



Figure 1: Relationship between projects, employees and customers

in which data is stored), in this column, we will restrict ourselves to using Python to store and retrieve data in database tables (both in memory, and in a file or in a SQL server). In this way, we will focus more on the DB layer and on access, and less on concepts like exposing a suitable middleware layer which could handle the same.

A brief introduction to an RDBMS

The entity relationship model was introduced by Peter Chen. It provided a mathematical basis for analysis and modelling of entities and relationships in a problem or solution domain. The concept of a relational database management system (RDBMS) was developed by E.F. Codd. He proposed 12 rules that needed to be followed by a database to be called a relational database. Over time, the term has been reduced to the more common shorter form—database—tending to represent only the storage and retrieval part of the RDBMS.

So what is an RDBMS? In its essence, the idea is to model entities in a problem or solution domain into tables that capture the most essential data related to that entity, which helps us to identify and model its most important functionality and behaviour. While the entities themselves get represented by *tables*, the association between entities could also be represented by *tables*. So a database is made up of a collection of tables which together represent the data that one wants to collect and organise, with respect to the domain that is being modelled in the solution.

To make the idea of a table more concrete, let us go back to the entities that we wrote about in the previous section. Each entity, project, employee and customer could be represented by a corresponding *table* in the *database*. In its simplest form, a project could be made up of the following attributes—project name and project code or ID. We could represent them simply, as shown in Figure 2, with 'Project Code' being the primary key of this entity. A primary key serves to identify instances of the entity, uniquely, within the system to be worked with. 'Project Name' could also serve as an alternate key in this case, as long as a project name is not reused.

There could be many more optional attributes, depending upon what is needed in the system, like project start date, project cost planned, project manager, etc, which we are not including for now.

An employee works on one or more projects at any given time. The same association can be modelled as shown in Figure 3. The combination of keys shown in the figure uniquely identifies this association (employee name, project

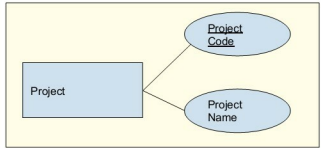


Figure 2: Details of project entity

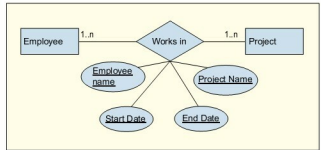


Figure 3: Association - 'works in'

name, start date and end date), as the same employee may be assigned to different projects at different times.

For the purpose of this column, let's simplify such an association with the following attributes, as shown in the table below, to make it easier to show simple examples of data entry and retrieval for the summary data stated in the problem statement.

Data label	Field label	Format
Date	Date	String
Employee Name	Empname	String
Project Name	Projname	String
Customer Name	Custname	String
Hours spent	Hours	Number



Note: The above table is neither optimised nor normalised. For the purposes of this column, however, this simplicity will be welcome.

A brief introduction to SQL

SQL or Structured Query Language is a specialised language used to work with RDBMSs. It has components that help define the structure of and create tables (CREATE statement), modify data in the tables (using INSERT, UPDATE, DELETE statements) and, most importantly, retrieve (SELECT) the data with syntax to filter records and manipulate results, as per needs. RDBMS systems are supposed to give a robust interface optimised for very fast data retrieval, even with a large number of records.

The SQL statement to create our table of interest, which we could call *workdata*, will look like what follows in SQLite3: